Davante

19

ENTORNOS DE DESARROLLO

Herramientas de control de versiones

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Características principales de las herramientas 4

/ 3. Estructura de las herramientas 4

/ 4. Caso práctico 1: "Distribuido o Centralizado" 5

/ 5. Git 6

5.1. Comandos básicos en ***Git*** 6

5.2. Otros comandos en Git 7

5.3. Ramas en Git 7

/ 6. SVN 7

/ 7. Caso práctico 2: "SVN vs Git" 8

/ 8. Resumen y resolución del caso práctico de la unidad 9

/ 9. Bibliografía 9

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de repositorio.

Distinguir los diferentes tipos de repositorios.

Conocer las principales características de Git.

Conocer las principales características de SVN.

/ 1. Introducción y contextualización práctica

Igual que sucede con todas las herramientas informáticas, aquellas que se destinan al desarrollo *software* se encuentran en constante evolución. En este tema nos vamos a centrar en los sistemas de control de versiones, pues existen diferentes aproximaciones para poder mantener el código sincronizado entre nuestro equipo de desarrollo. Gracias a los sistemas de control de versiones de código es posible aumentar la productividad del equipo de desarrollo.

Una de las razones es que podemos sincronizarnos fácilmente y detectar los cambios, así como mantener un registro de los diferentes cambios que han tenido lugar en el código. Existen diferentes tipos de enfoques a la hora de gestionar el código y los repositorios. En este tema nos centraremos en destacar las principales diferencias entre las herramientas más utilizadas en la actualidad.

Además, explicaremos el concepto de repositorio y las posibles aproximaciones para su implementación. Es necesario conocer el concepto de repositorio y cómo está implementado para poder comprender cómo funciona este tipo de herramienta y saber actuar en consecuencia cuando surjan problemas, por ejemplo: conflictos en el código a la hora de subirlo al repositorio.

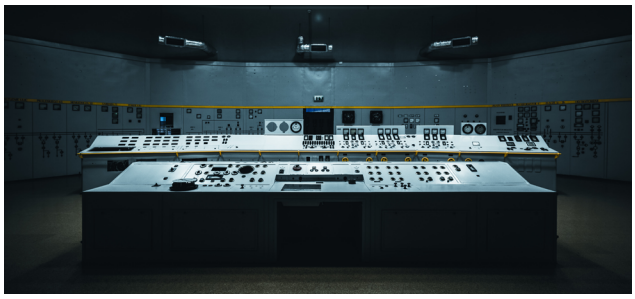


Fig. 1. Los repositorios de código pueden ser un punto crítico.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio Intro. "Utilización de ramas de desarrollo"

http://bit.ly/3q1ZP7G

/ 2. Características principales de las herramientas

Para poder implementar un **sistema de control de versiones** correctamente es necesario identificar los componentes que forman parte del **flujo de trabajo**. Normalmente, este tipo de sistemas requieren de una **aplicación local** que funcione como cliente y de otra **aplicación remota** que se comporte como servidor.

Siguiendo esta forma de organizarse, es posible trabajar colaborativamente teniendo en cuenta que pueden existir diferentes versiones del código y que al final van a terminar en una única versión para generar un producto determinado.

En la parte del servidor vamos a disponer de un **repositorio donde se ubicará el código fuente** y donde todos los clientes consultarán los cambios que se han ido realizando. Gracias al repositorio, todos los usuarios podrán sincronizarse de una manera centralizada.

Como se puede observar, no existe comunicación directa entre los clientes, sino que **la comunicación se realizará a través del repositorio de código**. Gracias a esto es posible guardar una **copia de seguridad del código en los repositorios y también una copia local**. Podríamos decir que un repositorio de código es un servidor de archivos, pero con una característica fundamental: poder guardar todos los cambios que se han ido realizando a lo largo del tiempo sobre cada uno de los archivos.

Además, los repositorios ofrecen diferentes **funcionalidades** para poder volver a dichas versiones, añadiendo la opción de comprobar los cambios que se han ido realizando. Cada vez que se realice un cambio en el repositorio, éste es capaz de guardar una descripción que nos indique el usuario que ha subido los cambios.

En algunos sistemas de gestión de versiones es necesario y obligatorio indicar dicho mensaje para poder guardar cambios en el repositorio. Como podemos intuir, cada herramienta de control de versiones posee su propio repositorio.

En ocasiones será posible también, establecer más de un repositorio de código con diferentes permisos de acceso para los diferentes roles de usuarios que tengamos en nuestra empresa. Al tratarse de un servidor, realizar este tipo de tareas es relativamente sencillo, pues solo habría que realizar una gestión adecuada de los roles.



Audio 1. "Requisitos hardware y resolución de conflictos"
http://bit.ly/3i1gMwj



/ 3. Estructura de las herramientas

Es posible realizar una **clasificación de los sistemas de control de versiones atendiendo a la forma de trabajo con sus repositorios:**

- **Distribuidos.** En este tipo de sistemas todos **los usuarios tienen su propio repositorio en la máquina local en la que están trabajando.**

  Gracias a esta forma de trabajo es posible mantener el **repositorio local sincronizado** incorporando una copia local del repositorio principal en cada una de las máquinas de los usuarios.

  Con este tipo de repositorio cambiar entre las diferentes ramas de desarrollo es muy sencillo, pues tenemos una copia total de repositorio en local.

Algunos de los sistemas de control de versiones más utilizados utilizan repositorios de este tipo, por ejemplo, *Git*.

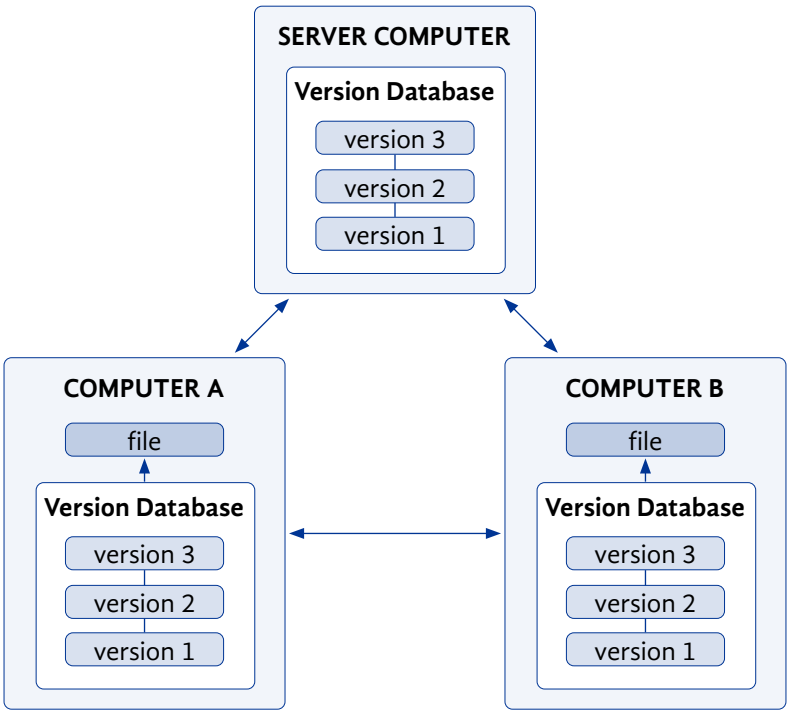


Fig. 2. Repositorio distribuido.

- **Centralizados.** En este caso solo **existe un repositorio centralizado que es controlado por un conjunto de usuarios**. Cuando descargamos del repositorio únicamente se descarga aquella parte necesaria para poder trabajar disminuyendo así la flexibilidad y posibilidades de trabajo. Se trata de un tipo de repositorio antiguo pero que se sigue utilizando en la actualidad. Un ejemplo de este tipo de repositorio es *Subversión* o SVN.

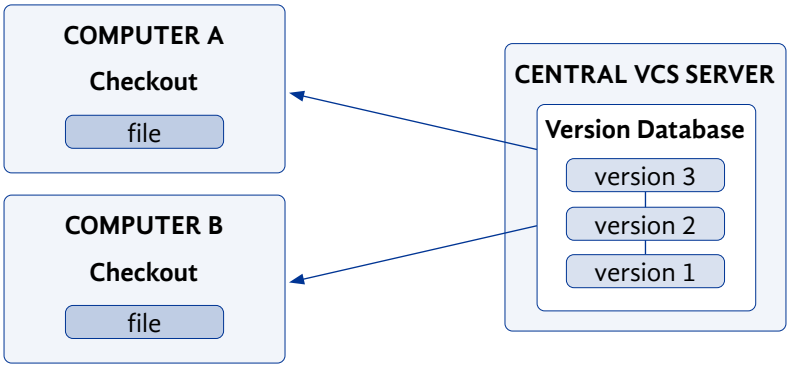


Fig. 3. Repositorio centralizado.

/ 4. Caso práctico 1: "Distribuido o Centralizado"

Planteamiento. En nuestro equipo de desarrollo necesitamos utilizar un sistema de control de versiones en el cual exista un mayor control del código, por lo que deseamos que éste esté siempre en el servidor. Dadas las características de seguridad de nuestra empresa, no podemos dispersar todo el código entre todos los desarrolladores.

Nudo. ¿Qué tipo de repositorio crees que sería más adecuado?

Desenlace. Según hemos visto anteriormente, en los repositorios que poseen una arquitectura distribuida existe menos control sobre el código en el que trabajamos. En este escenario, el código va a estar distribuido entre cada uno de los nodos que se conectan al repositorio de código.

Dadas las características de la empresa que se nos plantea en el caso de estudio, la mejor solución sería utilizar un repositorio centralizado para tener mayor control sobre el código fuente.

La característica de estos repositorios es que los usuarios solo podrán descargar el código que necesitan para poder trabajar mientras que el resto del código permanecerá en el repositorio.

Esta alternativa no suele ser la más utilizada en la actualidad, pero sí es cierto que en situaciones en las que prima la seguridad, puede ser necesario instaurar medidas de este tipo.

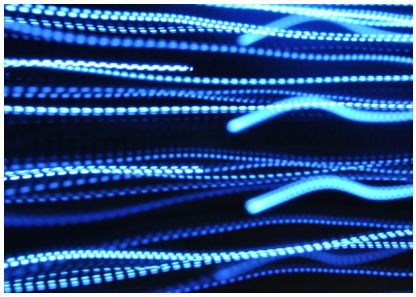


Fig. 4. La seguridad es un aspecto que puede afectar al código.

/ 5. Git

Git es uno de los **sistemas de control de versiones** más utilizados en la actualidad. Se trata de un **sistema distribuido** en el que **cada uno de los usuarios que trabaja en el proyecto posee una copia local de todo el repositorio**.

Gracias a esta copia es posible establecer un control de versiones local y luego realizar la integración en el repositorio remoto resolviendo las dependencias (en caso de que tengan lugar).

Como comprobaremos en los siguientes apartados, una de las formas más sencillas de trabajar en *Git* es mediante la utilización de **ramas de desarrollo.** A través de estas ramas es posible que cada desarrollador realice los cambios que estime oportunos y luego realice una petición de integración en la rama principal.

Nótese que, todos los proyectos tienen una rama principal que va a ser utilizada para mantener la línea fundamental de desarrollo o funcionalidades básicas, y a partir de ahí crear diferentes ramas para diferentes funcionalidades o momentos puntuales del desarrollo.

5.1. Comandos básicos en *Git*

El flujo de trabajo en *Git*, implicaría crear en primera instancia el repositorio donde queramos ubicar el código. A continuación, podríamos comenzar a trabajar con el **código fuente**. Cuando queramos guardar los cambios en el repositorio, utilizaremos los **comandos *add* y *commit*.**

Al utilizar estas órdenes, todos estos cambios se subirían al repositorio local de nuestra máquina, de tal forma que solo es visible por nosotros.

Si queremos subir los cambios al repositorio remoto tendríamos que utilizar la orden ***push*.** Normalmente, antes de realizar cualquier cambio en el repositorio remoto tenemos que traer los cambios ya existentes en el repositorio, pudiendo surgir algún tipo de conflicto. En el siguiente código se muestra el flujo de trabajo normal en *Git*.

```
1. git init
2. git add <filename>
3. git commit -m “Each time we commit a comment is needed”
4. git pull
5. git push
```

Código 1. Comandos del flujo de trabajo en Git.

5.2. Otros comandos en *Git*

Puede ser que, cuando nos incorporemos a un proyecto, éste haya sido iniciado anteriormente y lo que tenemos que realizar es una **clonación del código remoto a nuestra máquina**. Gracias a *Git* es posible realizar esto de una manera sencilla utilizando el siguiente **comando:**

```
1.  git clone username@host:/path/to/repo
```

Código 2. Clonación.

Si queremos visualizar los cambios que han tenido lugar en el repositorio podemos utilizar el **sistema de *logs* de *Git***. Utilizando el siguiente **comando** podemos ver los cambios que han tenido lugar en el repositorio:

```
1.  git log
```

Código 3. Log de cambios.

En el vídeo que se muestra a continuación, podrás observar el funcionamiento de *Git* empleando los comandos que hemos estudiado. Recuerda que debes tener un proyecto creado inicialmente para poder experimentar, y apreciar los cambios y actualizaciones que se pueden llevar a cabo con *Git*.

Vídeo 1. "Utilización de Git con interfaz gráfica"

http:/bit.ly/2lVBsg9

5.3. Ramas en *Git*

Como adelantamos en el tema anterior, al trabajar de forma colectiva sobre un mismo proyecto, pueden surgir algunos inconvenientes porque se originen muchos **conflictos a la hora de subir el código al repositorio.**

Una alternativa que se suele implementar es utilizar **ramas de desarrollo** para determinadas funcionalidades. De esta forma, diferentes equipos pueden trabajar sobre diferentes ámbitos de la aplicación.

Vídeo 2. "Utilización de ramas en Git"

http:/bit.ly/2ktoFkR

/ 6. SVN

Apache Subversion (SVN) es un **sistema de control de versiones centralizado que permite que diferentes usuarios puedan gestionar el código de una manera sencilla.**

Realmente, el funcionamiento de esta herramienta es mucho más sencillo que *Git* y, en su medida, también más limitado. Sin embargo, ofrece algunas **ventajas** como:

- Bloqueo de archivos.
- Creación de ramas y enlaces externos.
- Seguimiento efectivo de modificaciones realizadas.

A pesar de ello, algunas de las principales **desventajas** que podemos encontrar en esta herramienta es que en ocasiones es más complicado realizar las modificaciones en el repositorio, sobre todo cuando los nombres de los archivos cambian.

No hay que olvidar que el número de **sistemas operativos** que soportan de manera **nativa** este sistema de control de versiones cada vez es menor. Hay que tener en cuenta que se trata de un sistema de control de versiones bastante antiguo pero que se sigue utilizando por motivos de compatibilidad.

Al igual que sucede con *Git*, SVN ofrece una línea de comandos para poder trabajar de una manera sencilla y poder realizar *scripts,* y a la vez, diferentes interfaces gráficas que se pueden utilizar dependiendo del sistema operativo.

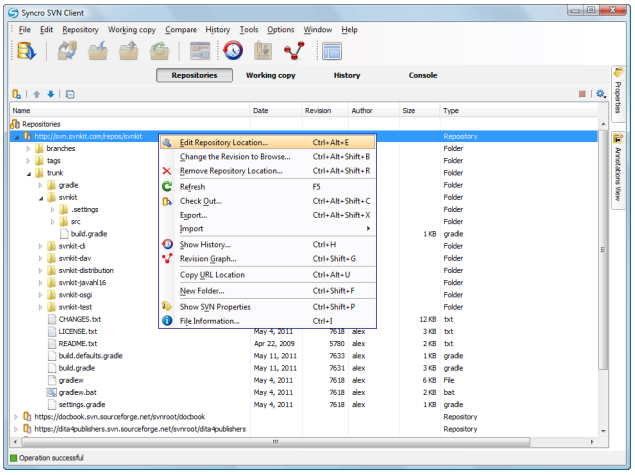


Fig. 5. Repositorio basado en SVN.

/ 7. Caso práctico 2: "SVN vs *Git*"

Planteamiento. Nuestro mejor cliente está interesado en que desarrollemos una nueva aplicación en el menor tiempo posible. Para ello, nuestra empresa nos provee de un equipo multidisciplinar para que, a través del entorno de desarrollo, podamos crear múltiples ramas y poder integrar fácilmente los cambios realizados. Para ello se nos ofrece un repositorio centralizado mediante SVN.

Nudo. ¿Crees que es una buena alternativa?

Desenlace. Según hemos visto en el tema, un repositorio centralizado tiene grandes problemas a la hora de trabajar con múltiples ramas si el número de desarrolladores aumenta significativamente. Como se nos plantea en el caso, es importante que tengamos una buena productividad ya que el cliente quiere que terminemos en el menor tiempo posible. Aparte del desarrollo, hay que considerar el tiempo de integraciones, pruebas, etc. Si en este contexto utilizamos una herramienta de control de versiones que no es adecuada y que nos puede requerir más tiempo para poder solventar los problemas que puedan surgir, puede que no lleguemos en la fecha prevista.

Dado que se nos ofrece un repositorio basado en SVN, lo recomendable en este caso sería utilizar un repositorio distribuido en el que todos los desarrolladores posean una copia del código en local y sea más fácil realizar las integraciones a posteriori. Por ello, una buena alternativa sería utilizar una herramienta como *Git*, ya que, aparte de ser una de las más utilizadas en la actualidad, utiliza un repositorio distribuido facilitando en gran medida el trabajo que se nos está pidiendo.

Nótese que, en ocasiones *Git* puede ser bastante complicado de utilizar si no tenemos la suficiente experiencia y el equipo desarrollo es relativamente grande.



Fig. 6. El desarrollo de software puede ser una carrera de fondo.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema hemos presentado los principios básicos para poder implementar un sistema de control de versiones, distinguiendo entre **centralizados y distribuidos**. Además, hemos conocido las principales características de las herramientas de control de versiones más utilizadas en la actualidad como ***Git*** y **SVN**. Como hemos visto, *Git* permite tener un repositorio distribuido entre los diferentes desarrolladores que tengamos en nuestro equipo.

Utilizando los diferentes **comandos** estudiados, será posible establecer y utilizar el sistema de control de versiones en nuestro desarrollo. En la actualidad, si trabajamos en un proyecto con múltiples desarrolladores, va a ser necesario que conozcamos los comandos básicos para cada sistema de gestión de versiones que incluyamos en el proyecto. Normalmente, vamos a tener un único sistema de gestión de versiones para el proyecto en el que estamos trabajando.

Sin embargo, puede ser que al cambiar de proyecto también cambie el sistema de gestión de versiones. Una alternativa a los comandos puede ser utilizar sistemas con interfaz gráfica que se integran en el sistema operativo. El problema de este tipo de programas es que solo están disponibles para determinados sistemas operativos, además, al cambiar de unos a otros, su interfaz gráfica varía, aunque, los comandos siguen siendo los mismos.

Resolución del caso práctico de la unidad

Según se nos plantea en el caso inicial, una buena solución para este problema sería crear diferentes ramas de desarrollo para cada una de las tareas que vamos a realizar, de esta forma, cada equipo tendrá su rama de desarrollo. Cuando cada equipo finalice la tarea que tiene asignada, solicitará una petición de integración en la rama principal. Esta rama tendrá tanto la solución corregida como la nueva funcionalidad que nos ha solicitado nuestro cliente. Aunque cada rama tiene que pasar sus propias pruebas de manera independiente, cuando realicemos la integración tendremos que repetir las pruebas para no cometer fallos de regresión.

/ 9. Bibliografía

Silverman, R. (2013). *Git pocket guide*. Sebastopol, CA: O'Reilly.

Gutiérrez, A. & Guervós, J. (2017). *Aprende Git*: [... y, de camino, GitHub. S.l: Independently Published.

Pilato, C., Sussman, B. & Fitzpatrick, B. (2008). *Version control with subversion*. Sebastopol, Calif: O'Reilly.